Laboratórne cvičenie 7

XAML, WPF a multiplatformová Avalonia UI

O čo ide

Dokončíte desktopovú aplikáciu **FRI DevPulse**, ktorá spája čítačku technologických a univerzitných RSS/Atom kanálov so správcom študijných a šprintových úloh. Na praktickom príklade si vyskúšate návrh rozhrania v XAML, oddelenie prezentačnej logiky podľa vzoru **MVVM (Model–View–ViewModel)** a prenos znalostí z **WPF** do multiplatformovej **Avalonia UI 12** na **.NET 10** (Windows, macOS aj Linux).

Precvičíte si obojsmerný Element Binding, CSS-like štýly a témy (`Dark` / `Light`), generovanie vlastností a príkazov cez `CommunityToolkit.Mvvm` (`[ObservableProperty]`, `[RelayCommand]`), prevod doménového stavu na farbu cez `IValueConverter`, hlásenie priebehu cez `IProgress<T>` so zrušením cez `CancellationToken` a automatické `xUnit` testy ViewModelu bez spúšťania okna.

Pracujte vlastným tempom. Orientačný plán je 100 minút (čas je odhad, riešenie neodovzdávate). AI asistentov aj internet môžete používať, pričom U1 a prvý príkaz v U2 odporúčame napísať ručne. Pred pokusom predpovedzte výsledok, prezrite zmeny v kóde a vysvetlite, čo sa stalo.

Ako je riešenie usporiadané

Projekt	Obsah	Úlohy
DevPulse.Core	Zdieľané modely, služby (`IFeedService`, `IDialogService`) a `MainWindowViewModel` bez UI závislostí	U2 a U3
DevPulse.App	Multiplatformový klient v Avalonia UI 12 (`.axaml` pohľady, témy, konvertory, DI v `App.axaml.cs`)	U1 až U3, Bonus
DevPulse.Tests	Automatické `xUnit` testy ViewModelu s falošnými (`Fake`) službami bez spúšťania okna	U4

Vzorové riešenie (`Solution/DevPulse.Windows.slnx`) navyše obsahuje projekt **DevPulse.Wpf** (`net10.0-windows`), ktorý pripája klasické WPF okno na ten istý `MainWindowViewModel` bez zmeny kódu v `DevPulse.Core`.

Plán práce

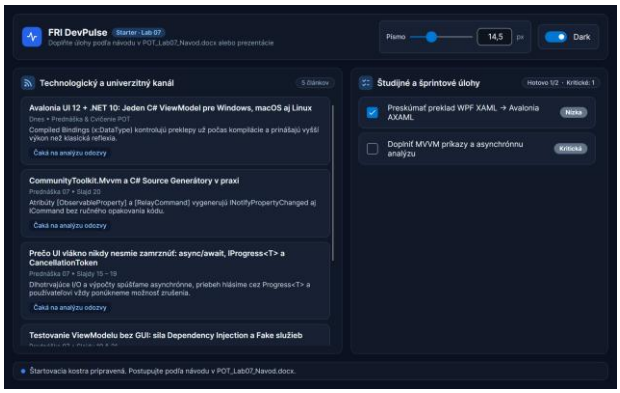
Blok	Čas	Výsledok
Štart a prehliadka	10 min	Riešenie `DevPulse.slnx` sa zostaví, okno sa zobrazí a zbehne 1 dodaný test
U1 Element binding a štýly	15 min	Posuvník mení písmo kartičiek, hover animácia a prepínanie tém Dark/Light
U2 Príkazy MVVM a konvertor	25 min	Pridávanie, mazanie a filtrovanie úloh, farebné odznaky priorít a `$parent[Window]`
U3 Asynchrónny kanál a analýza	25 min	Načítanie RSS/Atom kanála, živý `ProgressBar` cez `IProgress<T>` a tlačidlo Zastaviť
U4 Unit testy bez okna	15 min	`dotnet test` overí ViewModel cez `Fake` služby za zlomok sekundy + mutačná skúška
Rezerva	10 min	Ladenie v Avalonia DevTools (F12), vlastný experiment alebo voliteľný Bonus

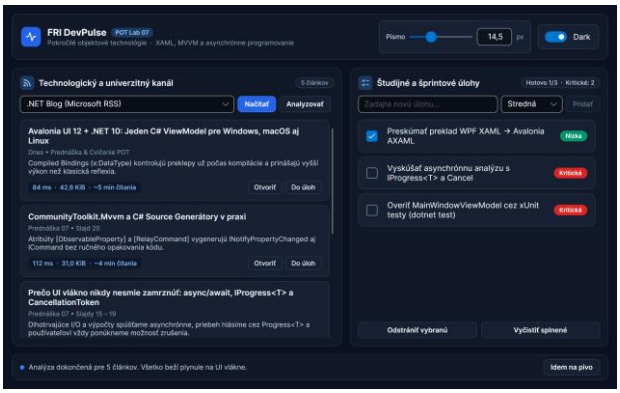
Postup pri každej úlohe: nájdite označené miesto `TODO`, doplňte kód podľa pravidiel v `AGENTS.md`, spustite aplikáciu alebo testy a porovnajte správanie s očakávaním. Ak v učebni vypadne internet, zvoľte v zozname zdrojov položku **Offline Demo – FRI UNIZA & POT**, ktorá funguje stopercentne lokálne.

Príprava projektov a spustenie

Jedno riešenie, tri projekty, päť upravovaných súborov. Rozbaľte archív `POT_Lab07_Starter_NET10.zip` do vlastného pracovného priečinka a otvorte súbor `DevPulse.slnx`. Projekt `DevPulse.App` je v riešení nastavený ako prvý spustiteľný projekt, takže vo **Visual Studiu 2026** funguje okamžite cez kláves `F5` alebo `Ctrl+F5` aj na počítačoch bez nainštalovaného rozšírenia pre návrhár Avalonia.

Potrebujete iba **.NET 10 SDK** (`10.0.x`) a ľubovoľné prostredie: Visual Studio 2026, JetBrains Rider, VS Code s rozšírením C# Dev Kit alebo editor s AI agentom. Všetky tri projekty cielia na `net10.0` so zapnutým `<Nullable>enable</Nullable>` a `<ImplicitUsings>enable</ImplicitUsings>`. Balíky `Avalonia` (12.x), `CommunityToolkit.Mvvm` (8.4+), `Microsoft.Extensions.Hosting` a `xunit` sú už v štartovacej kostre pripravené; prvé zostavenie obnoví závislosti z NuGet cache.





Vľavo: Štartovacia kostra Starter po prvom spustení. Vpravo: Dokončená aplikácia FRI DevPulse v tmavej téme.

Krátka prehliadka pred prácou: otvorte súbor `DevPulse.App/App.axaml.cs` a všimnite si, ako sa v metóde `OnFrameworkInitializationCompleted` skladá aplikácia cez `Host.CreateApplicationBuilder()` (Composition Root). Rozhrania `IFeedService`, `IDialogService` a `IBrowserLauncher` sa registrujú do kontajnera Dependency Injection a hlavnému oknu sa priradí `DataContext = Services.GetRequiredService<MainWindowViewModel>()`.

Prečítajte si aj súbor **AGENTS.md** v koreni riešenia (a stručný `.github/copilot-instructions.md`). Obsahuje záväzné pravidlá architektúry MVVM a odlišnosti Avalonia UI 12 oproti staršiemu WPF. Ak používate externý AI chat v prehliadači, pripojte mu na začiatku obsah `AGENTS.md`, aby vám negeneroval neexistujúce vlastnosti z WPF (napr. `Visibility="Collapsed"` alebo `MessageBox.Show`).

Súbor v štartovacom riešení	Čo je už pripravené	Čo doplníte
DevPulse.App/Views/MainWindow.axaml	Rozloženie okna, horná lišta, zoznamy kartičiek	U1 binding písma, U2 formulár a tlačidlá, U3 panel kanála a `ProgressBar`
DevPulse.App/App.axaml	Palety `Dark` a `Light`, vektorové ikony, štýly zoznamov	U1 plynulý prechod `Transitions` a hover stav `Border.card:pointerover`
DevPulse.Core/ViewModels/MainWindowViewModel.cs	Kolekcie, konštruktory, štatistiky a `OpenArticle`	U2 príkazy pre úlohy, U3 `LoadFeedAsync` a `AnalyzeFeedAsync`
DevPulse.App/Converters/PriorityToBrushConverter.cs	Konvertor popisov s diakritikou a kostra konvertora farieb	U2 prevod `TaskPriority` na zelenú, jantárovú a červenú kefu
DevPulse.Tests/MainWindowViewModelTests.cs	Falošné služby v `Fakes.cs` a 1 úvodný test inicializácie	U4 jednotkové testy príkazov, dialógu a zrušenia analýzy

VS Code, terminál a založenie projektu od nuly

Otvorenie vo VS Code alebo cez CLI: pri použití pripraveného ZIP archívu nič nové nezakladajte. Cez `File -> Open Folder` otvorte priečinok so súborom `DevPulse.slnx`, otvorte integrovaný terminál a overte SDK, testy aj spustenie okna:

```
dotnet --version
dotnet restore DevPulse.slnx
dotnet build DevPulse.slnx
dotnet test DevPulse.slnx
dotnet run --project DevPulse.App
```

Príkaz `dotnet build` preloží všetky tri projekty vrátane XAML pohľadov v `DevPulse.App`. Príkaz `dotnet test` v čistom starteri úspešne zbehne **1 dodaný test** (`Inicializacia_ObsahujePredvoleneZdrojeAUkazkoveUlohy`), prekladá však iba `DevPulse.Tests` a `DevPulse.Core` – chyby v `.axaml` súboroch (napr. `AVLN2000`) odhalí len `dotnet build` alebo `dotnet run`. Príkaz `dotnet run --project DevPulse.App` otvorí okno s dvomi ukážkovými úlohami a piatimi offline článkami. Po každej úprave XAML alebo C# aplikáciu ukončite a spustite znova (alebo vo Visual Studiu 2026 a Rideri využite Hot Reload).

Živá inšpekcia stromu prvkov (Avalonia DevTools): v súbore `DevPulse.App/Program.cs` je zaregistrovaný diagnostický nástroj `.WithDeveloperTools()` z balíka `AvaloniaUI.DiagnosticsSupport`. Keď je spustené okno aplikácie aktívne, stlačte kláves **F12** (na macOS **Fn+F12**). Otvorí sa inšpektor podobný nástrojom v prehliadači: môžete v ňom kliknúť na ľubovoľnú kartičku, sledovať aktuálne hodnoty `DataContext`, aktívne štýly, rozloženie okrajov (`Margin`, `Padding`) a vyhodnotené dynamické zdroje (`DynamicResource`).

Alternatíva: založenie Avalonia MVVM riešenia od nuly

Túto podčasť nevykonávajte, ak už pracujete v rozbalenom starteri; slúži ako prehľad na samostatné projekty a semestrálne práce. Šablóny Avalonia nainštalujete jednorazovo z NuGet a vygenerujete projekt s podporou `CommunityToolkit.Mvvm`:

```
dotnet new install Avalonia.Templates
dotnet new avalonia.mvvm -n MojaAplikacia -f net10.0 -m CommunityToolkit
```

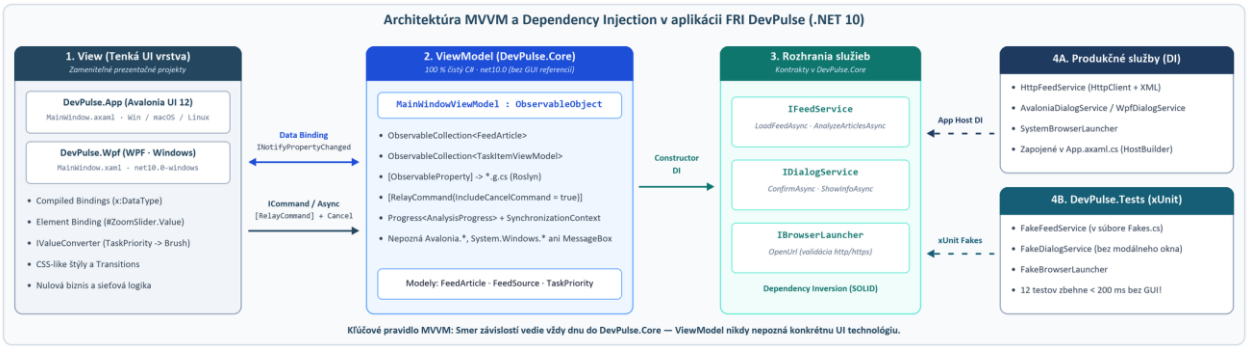
Ak chcete mať rovnaké čisté oddelenie vrstiev ako vo `FRI DevPulse`, presuňte priečinky `Models`, `Services` a `ViewModels` do samostatnej triednej knižnice (`dotnet new classlib -n MojaAplikacia.Core -f net10.0`) a pridajte k nej testovací projekt (`dotnet new xunit -n MojaAplikacia.Tests -f net10.0`). Tým vám už samotný kompilátor zabráni omylom použiť triedu `Window` alebo `SolidColorBrush` uprostred aplikačnej logiky.

Vo svojom `.csproj` projekte GUI vrstvy skontrolujte, že sú predvolene zapnuté kompilované väzby:

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  <AvaloniaUseCompiledBindingsByDefault>true</AvaloniaUseCompiledBindingsByDefault>
</PropertyGroup>
```

Čo sa pri oddelení View a ViewModelu deje

V tradičnom prístupe s udalosťami (`Click`, `TextChanged`) priamo v súbore `MainWindow.xaml.cs` skončí načítavanie siete, filtrovanie dát aj prepisovanie vlastností tlačidiel v jednej triede zviazanej s konkrétnym oknom. Takýto kód sa nedá spustiť v automatických testoch bez grafického subsystému a nedá sa znova použiť v inom klientovi. Vzor **MVVM** túto závislosť obracia: **View** pozná svoj **ViewModel**, ale ViewModel o existencii okna vôbec netuší.



Architektúra riešenia FRI DevPulse. Ten istý MainWindowViewModel v DevPulse.Core obsluhuje multiplatformové okno Avalonia UI 12, klasické WPF okno aj rýchle xUnit testy.

Oblasť	WPF (.NET 10 Windows)	Avalonia UI 12 (Win / macOS / Linux)
Súbor pohľadu a jadro	`.xaml`, DirectX + Windows API	`.axaml`, vlastný vykresľovací engine Skia
Dátové väzby (Bindings)	Klasické `{Binding}` vyhodnocované cez reflexiu za behu	Kompilované väzby s povinným `x:DataType` kontrolované pri builde
Element-to-Element väzba	`{Binding Value, ElementName=ZoomSlider}`	Stručný zápis `{Binding #ZoomSlider.Value}`
Skrývanie prvku v UI	`Visibility="Collapsed"` + `BooleanToVisibilityConverter`	Priama `bool` vlastnosť `IsVisible="{Binding IsBusy}"`
Štýly a prechody	`Style TargetType`, `Trigger` a `Storyboard`	CSS-like selektory (`Border.card:pointerover`) a `<Transitions>`

Kompilované väzby odhalia preklep ešte pred spustením. Vo WPF sa chybný zápis `{Binding Titulok}` preloží bez varovania a používateľ uvidí len prázdne pole (chyba sa schová v okne Output). Avalonia UI 12 vyžaduje na okne aj na každom `DataTemplate` atribút `x:DataType="..."`. Kompilátor vygeneruje priamy silne typovaný prístup k vlastnosti v C# a akýkoľvek preklep zastaví preklad s jasnou chybou **AVLN2000**.

Prečo `IDialogService` a `IValueConverter` namiesto `MessageBox` a `SolidColorBrush`? Ak by metóda `ClearCompletedTasksAsync` vo ViewModeli zavolala `MessageBox.Show(...)`, projekt `DevPulse.Core` by sa stal závislým od Windows knižnice a unit test v `DevPulse.Tests` by počas behu vyskočil s modálnym oknom a čakal na kliknutie človeka. Keď sa ViewModel pýta cez rozhranie `await _dialogService.ConfirmAsync(...)`, v Avalonii sa zobrazí modálne okno, vo WPF klasický `MessageBox` a v `xUnit` teste odpovie `FakeDialogService` v pamäti za mikrosekundu. Rovnako priorita úlohy ostáva vo ViewModeli čistým enumom `TaskPriority` a až konvertor vo vrstve View ju zmení na konkrétnu farbu.

Vlákno používateľského rozhrania je len jedno. Prvky okna smie meniť iba hlavné UI vlákno. Ak na ňom zavoláte synchrónne čakanie `.Result`, `.Wait()` alebo `Thread.Sleep()`, okno zamrzne a prestane sa prekresľovať. Keď služba na pozadí (`Task.Run` alebo asynchrónne HTTP volanie) potrebuje priebežne posúvať indikátor priebehu, hlási stav cez rozhranie `IProgress<T>`. Trieda `Progress<T>` vytvorená na začiatku príkazu si zapamätá `SynchronizationContext` UI vlákna a každý callback automaticky prepošle späť na hlavné vlákno.

U1 Obojsmerný Element Binding, štýly a téma

Cieľ: Prepojiť ovládacie prvky priamo v XAML bez jediného riadka v C#, overiť správanie kompilovaných väzieb (AVLN2000) a doplniť CSS-like hover animáciu kartičiek. Túto úlohu vypracujte **ručne bez AI**, aby ste získali istotu v syntaxi `.axaml`.

Kde: `DevPulse.App/Views/MainWindow.axaml` a `DevPulse.App/App.axaml`.

Postup – časť A (Element Binding v `MainWindow.axaml`):

1. V hornej lište nájdite komentár `TODO [Úloha 1]` pri posuvníku `x:Name="ZoomSlider"` (`Minimum="12"`, `Maximum="20"`). Hneď pod ním upravte prvok `<TextBox>` tak, aby sa jeho hodnota obojsmerne viazala na posuvník:

```
<TextBox Width="56"
         Text="{Binding #ZoomSlider.Value, Mode=TwoWay, UpdateSourceTrigger=LostFocus,
StringFormat={}{0:F1}}"
         HorizontalContentAlignment="Center"
         VerticalAlignment="Center" />
```

2. V tom istom súbore nájdite oba zoznamy `<ListBox>` (ľavý pre `models:FeedArticle` a pravý pre `vm:TaskItemViewModel`). V oboch šablónach pridajte k existujúcemu nadpisu `<TextBlock ... Text="{Binding Title}" ...>` atribút `FontSize="{Binding #ZoomSlider.Value}"`. Pri úlohe v pravom paneli **ponechajte `Grid.Column="1"`**, inak sa text vykreslí v nultom stĺpci cez zaškrtávacie políčko!

Postup – časť B (Štýl a animácia v `App.axaml`):

Na konci súboru `DevPulse.App/App.axaml` nájdite štýl `<Style Selector="Border.card">`. **Ponechajte všetkých 6 existujúcich `Setterov`** (zaoblenie, vnútorné odsadenie, rámik a pozadie karty), doplňte predvolenú transformáciu s prechodom `Transitions` a pod neho pridajte štýl pre stav `:pointerover`:

```
<Style Selector="Border.card">
    <Setter Property="CornerRadius" Value="10" />
    <Setter Property="Padding" Value="14,11" />
    <Setter Property="Margin" Value="0" />
    <Setter Property="BorderThickness" Value="1" />
    <Setter Property="BorderBrush" Value="{DynamicResource CardBorderBrush}" />
    <Setter Property="Background" Value="{DynamicResource CardBgBrush}" />
    <Setter Property="RenderTransform" Value="translateY(0px)" />
    <Setter Property="Transitions">
        <Transitions>
            <TransformOperationsTransition Property="RenderTransform" Duration="0:0:0.16" />
        </Transitions>
    </Setter>
</Style>

<Style Selector="Border.card:pointerover">
    <Setter Property="BorderBrush" Value="{DynamicResource BrandAccentBrush}" />
    <Setter Property="Background" Value="{DynamicResource CardHoverBgBrush}" />
    <Setter Property="RenderTransform" Value="translateY(-2px)" />
</Style>
```

Vyskúšajte a overte:

- Spustite aplikáciu. Posúvajte `ZoomSlider` – nadpisy článkov aj úloh sa plynulo zväčšujú a zmenšujú a `TextBox` vypisuje číslo s jedným desatinným miestom. Potom do `TextBox`u napíšte `18` a stlačte kláves `Tab` – posuvník skočí na hodnotu `18,0`.
- Prejdite myšou ponad kartičky a prepnite prepínač **Dark / Light** v pravom hornom rohu. Vďaka `DynamicResource` a slovníkom `ThemeDictionaries` v `App.axaml` sa okamžite prefarbia všetky panely, karty aj zvýraznenie okraja.
- **Krátky pokus s kompilátorom:** zmeňte dočasne v `MainWindow.axaml` zápis `{Binding Title}` na `{Binding Titulok}` a spustite `dotnet build`. Prečítajte si chybové hlásenie `AVLN2000` a vráťte názov späť na `Title`.

Zamyslite sa: Čo by sa stalo pri písaní čísla `18` do `TextBox`u, keby ste vynechali `UpdateSourceTrigger=LostFocus`? (V Avalonii má `TextBox.Text` predvolený trigger `PropertyChanged`, takže už po stlačení prvej číslice `1` by posuvník s `Minimum="12"` okamžite orezal hodnotu na `12,0`!)

U2 Príkazy MVVM, konvertor a šablóna položky

Cieľ: Spojazdniť pravý panel úloh a akčné tlačidlá na kartičkách článkov pomocou generátorov `CommunityToolkit.Mvvm` a vlastného konvertora `IValueConverter`. Prvý príkaz `AddTask` napíšte **ručne** a preskúmajte vygenerovaný kód; zvyšok bloku môžete dokončiť s pomocou AI asistenta.

Kde: `DevPulse.Core/ViewModels/MainWindowViewModel.cs`, `DevPulse.App/Converters/PriorityToBrushConverter.cs` a `DevPulse.App/Views/MainWindow.axaml`.

Krok 1: Prvý príkaz ručne a pohľad pod kapotu Source Generátora

V súbore `MainWindowViewModel.cs` pridajte nad pole `private string newTaskTitle = "";` atribút `[NotifyCanExecuteChangedFor(nameof(AddTaskCommand))]` a na miesto prvého `TODO [Úloha 2]` dopíšte:

```
private bool CanAddTask() => !string.IsNullOrWhiteSpace(NewTaskTitle);

[RelayCommand(CanExecute = nameof(CanAddTask))]
private void AddTask()
{
    var trimmed = NewTaskTitle.Trim();
    Tasks.Insert(0, CreateTask(trimmed, SelectedPriority));
    NewTaskTitle = "";
    StatusMessage = $"Pridaná úloha: „{trimmed}“.";
}
```

Zostavte projekt (`dotnet build`), nastavte kurzor na identifikátor `NewTaskTitle` alebo `AddTaskCommand` a stlačte **F12** (voľba „Go To Definition“, na macOS `Cmd+Click`). Otvorí sa vygenerovaný súbor `MainWindowViewModel.g.cs` (resp. `MainWindowViewModel.AddTask.g.cs`), kde uvidíte druhotnú časť triedy `partial class MainWindowViewModel` s volaniami `OnPropertyChanging`, `OnPropertyChanged` a lenivou inštanciou `RelayCommand`.

Čo napíšete vo ViewModeli	Čo vygeneruje kompilátor v `*.g.cs`	Na čo sa viaže View v `.axaml`
`[ObservableProperty] private string newTaskTitle = "";`	Verejnú vlastnosť `public string NewTaskTitle` s notifikáciou zmeny	`Text="{Binding NewTaskTitle}"`
`[RelayCommand(CanExecute = nameof(CanAddTask))] private void AddTask()`	Príkaz `public IRelayCommand AddTaskCommand`	`Command="{Binding AddTaskCommand}"`
`[RelayCommand] private async Task ClearCompletedTasksAsync()`	Asynchrónny `public IAsyncRelayCommand ClearCompletedTasksCommand`	`Command="{Binding ClearCompletedTasksCommand}"`

Krok 2: Zvyšné príkazy úloh a dialóg cez rozhranie

Doplňte do `MainWindowViewModel.cs` nad pole `selectedTask` atribút `[NotifyCanExecuteChangedFor(nameof(RemoveSelectedTaskCommand))]` a pridajte tri zostávajúce príkazy pre prácu so zoznamom úloh:

- `AddArticleToTasks(FeedArticle? article)` – ak `article` nie je `null` a úloha s názvom `$"Prečítať: {article.Title}"` ešte v `Tasks` neexistuje, vloží ju na začiatok so strednou prioritou a odkazom `sourceUrl: article.Link`.
- `RemoveSelectedTask()` s `CanExecute = nameof(CanRemoveSelectedTask)` (`SelectedTask is not null`) – vymaže označenú úlohu z `Tasks` a vynuluje `SelectedTask = null`.
- `ClearCompletedTasksAsync()` – ak existujú splnené úlohy, vypýta si potvrdenie cez `var confirmed = await _dialogService.ConfirmAsync("Vyčistiť splnené úlohy", ...);`. Ak používateľ zvolí „Zrušiť“ (`!confirmed`), metóda len vypíše oznam do `StatusMessage` a skončí bez mazania!

Krok 3: Konvertor farieb a prepojenie XAML (`$parent[Window]`)

1. V súbore `DevPulse.App/Converters/PriorityToBrushConverter.cs` nahraďte telo metódy `Convert` výrazom `switch`, ktorý pre `TaskPriority.Nizka` vráti zelenú kefu `#059669`, pre `TaskPriority.Stredna` jantárovú `#D97706`, pre `TaskPriority.Kriticka` červenú `#DC2626` a pre iné hodnoty `DefaultBrush` (`#64748B`). Kód kefy si predpripravte do statických polí `static readonly IBrush`, aby sa pri každom prekreslení nealokovali nové objekty.

2. V `MainWindow.axaml` doplňte podľa komentárov `TODO [Úloha 2.1]`:

- V pravom paneli vstupný riadok (`TextBox` s `PlaceholderText="Zadajte novú úlohu…"`, `ComboBox` priorít a tlačidlo `Pridať` s `Classes="accent"` a `IsDefault="True"`) a do pripraveného `<UniformGrid DockPanel.Dock="Bottom">` dvojicu tlačidiel `Odstrániť vybranú` a `Vyčistiť splnené`. Dbajte na to, aby `<ListBox>` zostal v `<DockPanel>` zapísaný ako **úplne posledný potomok**!
- V ľavom paneli na kartičke článku majú položky `DataTemplate` typ `x:DataType="models:FeedArticle"`, no príkazy `OpenArticleCommand` a `AddArticleToTasksCommand` patria hlavnému ViewModelu okna. Prevolajte ich cez typovaný rodičovský kontext:

```
<Button Grid.Column="1"
        Classes="subtle"
        Content="Otvoriť"
        Command="{Binding
$parent[Window].((vm:MainWindowViewModel)DataContext).OpenArticleCommand}"
        CommandParameter="{Binding Link}" />
<Button Grid.Column="2"
        Classes="subtle"
        Content="Do úloh"
        Command="{Binding
$parent[Window].((vm:MainWindowViewModel)DataContext).AddArticleToTasksCommand}"
        CommandParameter="{Binding}" />
```

Vyskúšajte a overte: tlačidlo `Pridať` je pri prázdnom poli či medzerách neaktívne, po napísaní názvu sa rozsvieti a stlačenie `Enter` pridá úlohu s farebným odznakom (`Nízka`, `Stredná`, `Kritická`). Tlačidlo `Do úloh` prenesie článok z ľavého zoznamu do pravého (druhé kliknutie na ten istý článok nevytvorí duplikát) a `Vyčistiť splnené` otvorí modálne potvrdzovacie okno.

Zamyslite sa: Prečo musí byť trieda `MainWindowViewModel` označená kľúčovým slovom `partial`? Prečo je položka úlohy `TaskItemViewModel` zaradená do `ViewModels` (napojená na prepočet štatistík okna), kým `FeedArticle` leží v priečinku `Models`?

U3 Asynchrónny kanál, IProgress<T> a zrušenie

Cieľ: Doplniť asynchrónne sťahovanie RSS/Atom zdrojov a priebežnú analýzu článkov tak, aby okno nikdy nezamrzlo, používateľ videl plynulý `ProgressBar` a mohol bežiacu analýzu kedykoľvek bezpečne prerušiť tlačidlom `Zastaviť`.

Kde: `DevPulse.Core/ViewModels/MainWindowViewModel.cs` a `DevPulse.App/Views/MainWindow.axaml`.

Postup vo ViewModeli (`MainWindowViewModel.cs`):

1. Pridajte nad pole `selectedSource` atribút `[NotifyCanExecuteChangedFor(nameof(LoadFeedCommand))]` a nad pole `isBusy` dvojicu atribútov `[NotifyCanExecuteChangedFor(nameof(LoadFeedCommand))]` a `[NotifyCanExecuteChangedFor(nameof(AnalyzeFeedCommand))]`.

2. Implementujte príkaz `LoadFeedAsync(CancellationToken cancellationToken)` s atribútom `[RelayCommand(CanExecute = nameof(CanLoadFeed), IncludeCancelCommand = true)]`, kde `CanLoadFeed()` kontroluje `SelectedSource is not null && !IsBusy`. Pred volaním `await _feedService.LoadFeedAsync(SelectedSource.Url, cancellationToken)` nastavte `IsBusy = true`, v bloku `catch` ošetrite `OperationCanceledException` aj bežnú sieťovú `Exception` (odporučte prepnúť na Offline Demo zdroj) a v bloku `finally` vždy vráťte `IsBusy = false` a zavolajte `AnalyzeFeedCommand.NotifyCanExecuteChanged()`.

3. Implementujte príkaz `AnalyzeFeedAsync(CancellationToken cancellationToken)` s `[RelayCommand(CanExecute = nameof(CanAnalyzeFeed), IncludeCancelCommand = true)]`, kde `CanAnalyzeFeed()` vráti `Articles.Count > 0 && !IsBusy`. Keďže `Progress<T>` posiela hlásenia asynchrónne (a v testoch bez okna cez `ThreadPool`), chráňte aktualizáciu stavu lokálnym zámkom a príznakom `isActive`, aby žiadny oneskorený callback neprepísal záverečnú správu:

```
var snapshot = Articles.ToList();
var gate = new object();
var isActive = true;
IsBusy = true;
ProgressPercent = 0;

IProgress<AnalysisProgress> progress = new Progress<AnalysisProgress>(p =>
{
    lock (gate)
    {
        if (!isActive || cancellationToken.IsCancellationRequested)
            return;

        ProgressPercent = p.Percent;
        StatusMessage = $"Analyzujem ({p.Completed}/{p.Total}): {p.CurrentTitle}";
    }
});
```

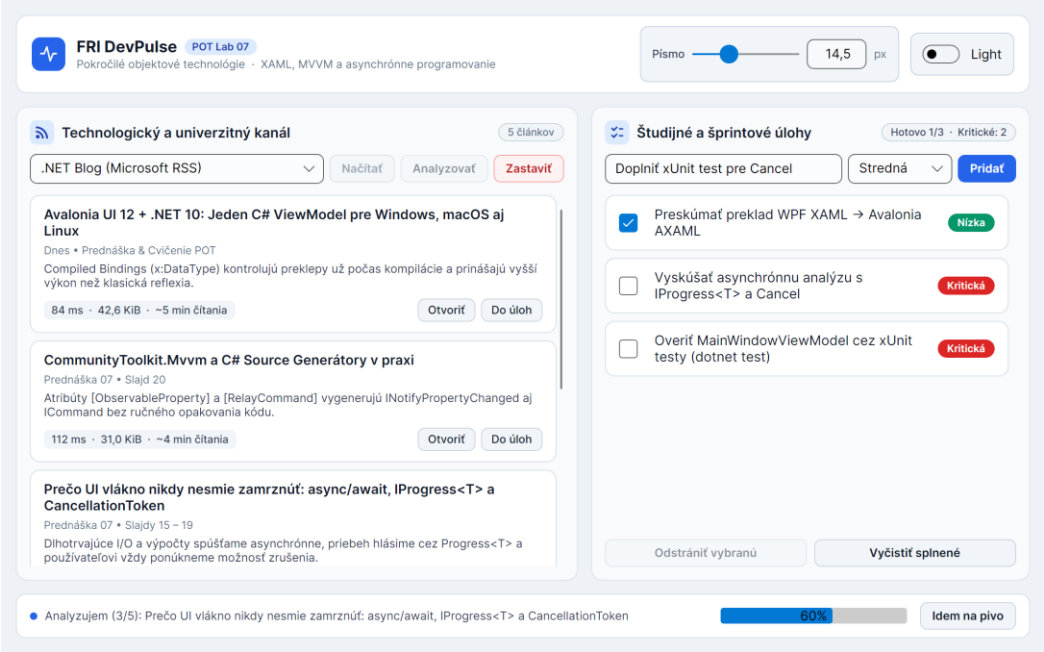
V bloku `try` zavolajte `await _feedService.AnalyzeArticlesAsync(snapshot, progress, cancellationToken)`, pred nastavením záverečnej správy pod tým istým `lock (gate)` zhoďte `isActive = false`, zachyťte `OperationCanceledException` (správa s textom `"Analýza bola zastavená používateľom."`) a vo `finally` vráťte `IsBusy = false`.

Zapojenie ovládacích prvkov a priebehu vo View (`MainWindow.axaml`)

1. Pod nadpis ľavého panela (`TODO [Úloha 3.1]`) vložte ovládací `<Grid ColumnDefinitions="*,Auto,Auto,Auto" ColumnSpacing="6">` s výberom zdroja `ComboBox` (`ItemsSource="{Binding Sources}"`, `SelectedItem="{Binding SelectedSource}"`), tlačidlom `Načítať` (`Classes="accent"`, `LoadFeedCommand`), tlačidlom `Analyzovať` (`AnalyzeFeedCommand`) a červeným tlačidlom `Zastaviť` (`Classes="danger"`, `Command="{Binding AnalyzeFeedCancelCommand}"`, `IsVisible="{Binding AnalyzeFeedCommand.IsRunning}"`).

2. Do spodného stavového riadka doplňte indikátor priebehu s povinným `Grid.Column="1"` a šírkou `Width="200"`:

```
<ProgressBar Grid.Column="1"
             Width="200"
             Minimum="0" Maximum="100"
             Value="{Binding ProgressPercent}"
             ShowProgressText="True"
             IsVisible="{Binding IsBusy}"
             VerticalAlignment="Center" />
```



Svetlá téma počas behu asynchrónnej analýzy. Tlačidlá Načítať a Analyzovať sú uzamknuté cez CanExecute, zobrazuje sa tlačidlo Zastaviť a spodný riadok ukazuje živý priebeh.

Vyskúšajte a overte: načítajte online kanál (napr. „.NET Blog“ alebo „Živé.sk“) alebo lokálny zdroj „Offline Demo“. Kliknite na `Analyzovať`, počas behu hýbte oknom alebo prepnite tému – rozhranie zostáva plynulé. Pri druhom spustení kliknite v polovici na `Zastaviť`: analýza okamžite skončí bez pádu aplikácie a tlačidlá sa opäť odomknú.

Zamyslite sa: Odkiaľ sa vzala vlastnosť `AnalyzeFeedCancelCommand`, keď ste takú metódu nikde nenapísali? Prečo používame `IsVisible="{Binding IsBusy}"` priamo bez konvertora `BooleanToVisibilityConverter`?

U4 Automatické testy ViewModelu bez GUI

Cieľ: Dokázať hlavný prínos vzoru MVVM – otestovať kompletné správanie `MainWindowViewModel` automatizovanými `xUnit` testami bez vytvárania okna a vykonať mutačnú skúšku, ktorá overí, že test skutočne zachytí chybu.

Kde: `DevPulse.Tests/MainWindowViewModelTests.cs` (falošné implementácie služieb si prezrite v súbore `DevPulse.Tests/Fakes.cs`).

Postup: V súbore `MainWindowViewModelTests.cs` doplňte minimálne 4 testovacie metódy označené `[Fact]`:

1. **Validácia vstupu a CanExecuteChanged:** overte, že pri `vm.NewTaskTitle = "  "` vráti `vm.AddTaskCommand.CanExecute(null)` hodnotu `false`, a po priradení platného textu sa vyvolá udalosť `CanExecuteChanged` a `CanExecute` vráti `true`.

2. **Pridanie úlohy:** po zavolaní `vm.AddTaskCommand.Execute(null)` pribudne nová úloha na index `0` kolekcie `vm.Tasks` so zvolenou prioritou a `vm.NewTaskTitle` sa vyprázdni.

3. **Potvrdenie verzus zamietnutie dialógu:** vytvorte `var dialogs = new FakeDialogService { NextConfirmationResult = false };` a po `await vm.ClearCompletedTasksCommand.ExecuteAsync(null)` overte, že počet úloh v `vm.Tasks` zostal nezmenený (`dialogs.ConfirmCallsCount == 1`). V druhom teste s `NextConfirmationResult = true` overte, že splnené úlohy zmizli (`Assert.DoesNotContain(vm.Tasks, t => t.IsCompleted)`).

4. **Zrušenie asynchrónnej analýzy:** nastavte `new FakeFeedService { SimulatedAnalysisDelay = TimeSpan.FromMilliseconds(250) }`, spustite analýzu bez okamžitého `await` (`var running = vm.AnalyzeFeedCommand.ExecuteAsync(null);`), po `await Task.Delay(50)` overte uzamknutie `Assert.False(vm.LoadFeedCommand.CanExecute(null))`, zavolajte `vm.AnalyzeFeedCancelCommand.Execute(null)` a po `await running;` overte `Assert.False(vm.IsBusy)` a odomknutie príkazu.

```
dotnet test DevPulse.slnx
```

Povinná mutačná skúška (kontrola proti falošne zeleným testom): v metóde `ClearCompletedTasksAsync` dočasne zakomentujte celú vetvu `if (!confirmed)` (pri blokovom zápise vrátane `{ ... }`, nie iba riadok s podmienkou) a spustite `dotnet test`. Test pre zamietnutý dialóg **musí sčervenať**! Potom zmenu vráťte a overte, že sú testy opäť zelené.

Keď sa nedarí

- **Do poľa veľkosti písma nejde napísať `18` (skočí na `12,0`):** vo väzbe na `#ZoomSlider.Value` chýba `UpdateSourceTrigger=LostFocus`. Po dopísaní čísla stlačte `Tab`.
- **Nadpis úlohy v pravom zozname prekrýva zaškrtávacie políčko:** pri pridávaní `FontSize` ste omylom vymazali `Grid.Column="1"` na prvku `<TextBlock Text="{Binding Title}">`.
- **Chyba prekladu `AVLN2000`:** skontrolujte presný názov vlastnosti vo ViewModeli (vrátane prípony `Command`), prítomnosť `x:DataType` a pri tlačidlách v šablóne článku pretypovanie `$parent[Window].((vm:MainWindowViewModel)DataContext)`.
- **Zoznam sa nerozťahuje alebo nevidno spodné tlačidlá:** v `<DockPanel>` rozhoduje poradie zápisu – prvky s `DockPanel.Dock="Top"` a `DockPanel.Dock="Bottom"` musia byť zapísané pred záverečným `<ListBox>`.
- **V stavovom riadku sa `ProgressBar` prekrýva s textom:** chýba mu atribút `Grid.Column="1"`.

Vlastná kontrola a rozšírenia

Viete bez nápovedy ukázať vo vygenerovanom `*.g.cs` súbore, čo vytvoril atribút `[RelayCommand]`, vysvetliť prečo `DevPulse.Core` nepozná `Avalonia.Media.IBrush`, a ukázať v kóde presné miesto, kde sa `CancellationToken` prenáša od tlačidla `Zastaviť` až po cyklus merania článkov?

Pre rýchlejších: pridajte nad pravý zoznam úloh vyhľadávacie pole `FilterText` alebo prepínač „Iba nesplnené“, prípadne doplňte do `TaskItemViewModel` otváranie uloženého `SourceUrl` pri úlohách vytvorených z článkov a pokryte novú funkciu vlastným `xUnit` testom.

AI ako pomocník pri implementácii a učení

Prompty nižšie môžete vložiť do AI asistenta vo svojom editore (Copilot, Antigravity, Cursor) alebo do samostatného chatu. Vždy pripojte súbor `AGENTS.md` a iba tie súbory, ktorých sa týka aktuálny krok. Najprv si nechajte vysvetliť návrh zmien, až potom kód prijmite a overte spustením.

Najprv pochopiť architektúru projektu

Prompt: „Pracujem na cvičení Lab 07 (`FRI DevPulse`) v `.NET 10` a Avalonia UI 12. Pozri si `DevPulse.slnx`, `AGENTS.md`, `App.axaml.cs` a všetky tri `.csproj` súbory. Bez úpravy kódu mi stručne vysvetli: 1. ako sú rozdelené zodpovednosti medzi `DevPulse.Core`, `DevPulse.App` a `DevPulse.Tests`, 2. prečo `DevPulse.Core` nemá referenciu na Avalonia a ako potom ViewModel zobrazí dialóg alebo zafarbí prioritu, 3. čo robí `AvaloniaUseCompiledBindingsByDefault`. Na záver mi polož dve kontrolné otázky a počkaj na moju odpoveď."

Jeden kontrolovaný krok (U2 – dokončenie TODO modulu)

Prompt pre U2: „Prečítaj si `AGENTS.md`, `MainWindowViewModel.cs`, `PriorityToBrushConverter.cs` a `MainWindow.axaml`. Príkaz `AddTask` už mám hotový. Navrhni iba zmeny pre U2: príkazy `AddArticleToTasks`, `RemoveSelectedTask` a `ClearCompletedTasksAsync` (cez `_dialogService.ConfirmAsync`), konvertor farieb priorít (`#059669`, `#D97706`, `#DC2626`) a XAML prvky pre TODO panel a tlačidlá kartičiek cez `$parent[Window]`. Neimplementuj zatiaľ U3 ani Bonus. Zachovaj poradie prvkov v `DockPanel` a nepoužívaj žiadne emoji znaky."

Váš krok: skontrolujte v navrhnutom XAML, či `<ListBox>` zostal posledným prvkom v `<DockPanel>` a či pri `ClearCompletedTasksAsync` nezmizla vetva `if (!confirmed)`, ktorá pri zamietnutí ukončí metódu ešte pred mazaním.

Asynchrónnosť a vláknová bezpečnosť (U3)

Prompt pre U3: „Podľa pravidiel v `AGENTS.md` doplň do `MainWindowViewModel.cs` a `MainWindow.axaml` úlohu U3: príkazy `LoadFeedAsync` a `AnalyzeFeedAsync` so zrušením (`IncludeCancelCommand = true`), hlásením priebehu cez `Progress<AnalysisProgress>` odovzdané do `_feedService.AnalyzeArticlesAsync` a zobrazením `ProgressBar` (`Grid.Column="1"`) aj tlačidla `Zastaviť`. Vysvetli mi, prečo v callbacku `Progress<T>` používame zámok `lock (gate)` s príznakom `isActive` pri behu `xUnit` testov bez `SynchronizationContext`."

Diagnostika chyby podľa dôkazov

Prompt pri chybe: „Pri úlohe [číslo] očakávam [očakávané správanie], ale dostávam [presný text chyby z buildu alebo opis správania]. Prikladám `AGENTS.md` a aktuálny úsek `MainWindow.axaml` / `MainWindowViewModel.cs`. Zoraď tri najpravdepodobnejšie príčiny, navrhni jeden overovací pokus (napr. cez Avalonia DevTools `F12` alebo `dotnet test`) a uprav iba chybné miesto bez prepisovania zvyšku súboru."

Učenie formou tútora bez hotového kódu

Prompt: „Buď môj tútor k Lab 07 (MVVM, WPF a Avalonia UI 12). Pýtaj sa ma po jednej otázke: rozdiel medzi reflexiou vo WPF a `x:DataType` v Avalonii, úloha kľúčového slova `partial` pri `CommunityToolkit.Mvvm`, rozdiel medzi `UpdateSourceTrigger=PropertyChanged` a `LostFocus` pri číselnom posuvníku, prenos `Progress<T>` na UI vlákno a význam mutačnej skúšky v `xUnit` testoch. Vždy počkaj na moju odpoveď a nedávaj mi riešenie dopredu."

Voliteľný príklad: utekajúce tlačidlo a porovnanie s WPF

Tento blok rozširuje základné cvičenie o dve ukážky nadväzujúce na prednášku 07: animáciu tlačidla „Idem na pivo“ prepojenú so stavom ViewModelu a porovnanie s natívnym **WPF klientom** nad identickým jadrom.

Bonus: tlačidlo „Idem na pivo“ (prepojenie View animácie s MVVM stavom)

Na prednáške tlačidlo pri priblížení kurzora uskakovalo v code-behind bez ohľadu na stav úloh. V MVVM aplikácii **ViewModel rozhoduje o doménovom pravidle** (`HasPendingCriticalTasks`) a **View rieši iba grafický posun** (`DoubleTransition`):

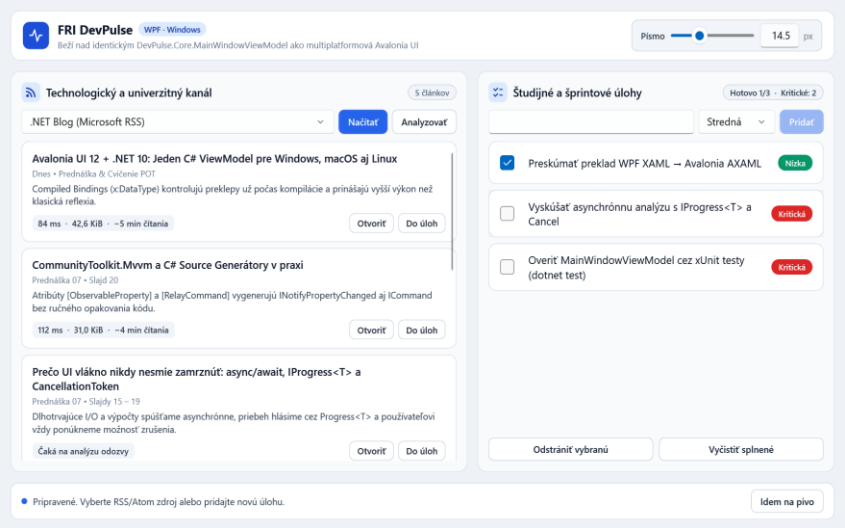
1. Do `MainWindowViewModel.cs` doplňte `public bool HasPendingCriticalTasks => PendingCriticalCount > 0;` a príkaz `[RelayCommand] private async Task GoForBeerAsync()` volajúci `_dialogService.ShowInfoAsync(...)`.

2. Do stavového riadka v `MainWindow.axaml` (`Grid.Column="2"`) pridajte tlačidlo `x:Name="BeerButton"` (`Content="Idem na pivo"`, `Command="{Binding GoForBeerCommand}"`, `PointerEntered="BeerButton_OnPointerEntered"`) s `<TranslateTransform X="0">` a `<DoubleTransition Property="X" Duration="0:0:0.18" Easing="CubicEaseOut" />`.

3. V `MainWindow.axaml.cs` pri `HasPendingCriticalTasks == true` prepínajte `transform.X` medzi `-130` a `0`; po odškrtnutí kritických úloh vráťte `transform.X = 0`.

Jeden ViewModel, dva desktopové svety (Avalonia UI 12 verzus WPF)

Na systéme Windows si vo vzorovom riešení `Solution/DevPulse.Windows.slnx` spustite vedľa seba `DevPulse.App` aj `DevPulse.Wpf`:



Natívny Windows klient DevPulse.Wpf spustený nad tým istým projektom DevPulse.Core bez zmeny vo ViewModeli.

Porovnajte `DevPulse.Wpf/MainWindow.xaml` a `DevPulse.App/Views/MainWindow.axaml`: WPF používa `BooleanToVisibilityConverter`, `ElementName=ZoomSlider` a `RelativeSource AncestorType=Window`, kým Avalonia používa `IsVisible`, `#ZoomSlider.Value` a `$parent[Window]`. Triedy `MainWindowViewModel`, `HttpFeedService` aj všetkých 12 testov v `DevPulse.Tests` sú na 100 % spoločné.

Dokumentácia na ďalšie štúdium

- Avalonia UI 12 dokumentácia: https://docs.avaloniaui.net/
- CommunityToolkit.Mvvm a Source Generátory: https://learn.microsoft.com/dotnet/communitytoolkit/mvvm/
- WPF v .NET 10 (Fluent štýly a XAML): https://learn.microsoft.com/dotnet/desktop/wpf/
- .NET 10 SDK a formát riešení `.slnx`: https://learn.microsoft.com/dotnet/core/compatibility/sdk/10.0/dotnet-new-sln-slnx-default